

Vind de volgorde

- **Tijdslimiet:** 10 minuten voor het hele bestand
- **Omgeving:** één GPU (≈16 GB VRAM), geen internet
- **Grootte van de oplossing:** `solution.ipynb` ≤ 1 MB
- **Opslag:** 5 GB

Probleem

Je bent Engelstalige telefoongesprekken aan het (af)luisteren. Deze dialogen zijn tussen twee deelnemers, *Spreker A* en *Spreker B*. Elke dialoog is opgedeeld in spreekbeurten, waarbij slechts één spreker aan het spreken is per beurt. Elke beurt wordt opgeslagen als een afzonderlijk `.wav`-audiobestand, zodat een volledige dialoog wordt weergegeven door een verzameling `.wav`-bestanden, één voor elke beurt.

Helaas zijn door slecht management bij BelMoeilijk bv de beurten willekeurig door elkaar geschud, waardoor het gesprek niet langer logisch is. In de bestandsnaam `chunk_{k}.wav` verwijst k naar het k -de fragment in de door elkaar geschudde verzameling, niet naar de k -de beurt in de oorspronkelijke dialoog.

!! Het is jouw taak om de oorspronkelijke chronologische volgorde van het gesprek te reconstrueren.



Dataset

Elke dialoog bevat n -audiobestanden met de namen `chunk_0.wav`, `chunk_1.wav`, ..., `chunk_{n-1}.wav`. De fragmenten zijn afzonderlijke beurten. De bestandsnamen komen alleen overeen met de door elkaar geschudde volgorde. Ze geven niet aan waar een fragment in het oorspronkelijke gesprek thuishoort. Elke dialoog heeft 7–20 fragmenten, mono, 44.1 kHz (je mag resamplen als dat nodig is voor een model).

prefix.json bevat de bestandsnaamindices van de eerste twee fragmenten in elke dialoog. Hiermee wordt het werkelijke begin van de dialoog aangegeven en wordt de ambiguïteit tussen het voorwaarts of achterwaarts lezen van het gesprek weggenomen.

Bijvoorbeeld: 11: [7, 12] betekent dat de eerste en tweede beurt van dialoog 11 respectievelijk `chunk_7.wav` en `chunk_12.wav` zijn.

Ontvangsten (als het goed is)

Je ontvangt **twee mappen met dezelfde indeling**:

Map	Dialogen	answers.json?	Gebruik deze om
dataset/train/	1,288	✓ inbegrepen	je model te trainen / fine-tunen
dataset/test_public/	100	✓ inbegrepen	je pipeline uit te voeren en de score lokaal zelf te berekenen

Tijdens de beoordeling wordt je map `dataset/test_public/` vervangen door een hidden evaluation set (`test_leaderboard_a` voor het openbare leaderboard en `test_leaderboard_b` voor het definitieve leaderboard) — deze hebben dezelfde grootte en indeling als `dataset/test_public/`, maar zonder `answers.json`.

Je notebook wordt opnieuw op die data uitgevoerd en het bestand `answers.json` dat het produceert, wordt gebruikt voor de beoordeling. De apart gehouden testdialogen komen uit dezelfde verdeling als `train`, dus je lokale `test_public`-score is een betrouwbare indicatie.

Mappenstructuur

```
dataset/train/
  prefix.json # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P} ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

Uitvoer

Bepaal voor elke dialoog de oorspronkelijke chronologische volgorde van de audiofragmenten. Je voorspelling moet een permutatie P van $\{0, 1, \dots, n-1\}$ zijn, waarbij $P[i]$ de voorspelde chronologische positie van `chunk_i.wav` is (0 = eerste).

Je uitvoerbestand `answers.json` moet elke dialoog-ID koppelen aan de voorspelde permutatie (volgorde):

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5],
  "niet toegestaan": [0, 0, 1, 2],
  "niet toegestaan": "en toen zei Romeo tegen Julia: 'Ik hou van je :3'",
  "niet toegestaan": [5, 2, 1, 3]
  "niet toegestaan": "en toen zei Julia tegen Romeo: 'Sure! Ik ga mijn eigen dood faken'"
  "en toen": "stierf Romeo van liefdesverdriet"
  "error": komma's moeten niet vergeten worden
}
```

Voorbeeld

Een dialoog heeft 3 door elkaar geschudde fragmenten `chunk_0`, `chunk_1`, `chunk_2`:

door elkaar geschud fragment	gesproken inhoud	werkelijke positie (rang)
<code>chunk_0.wav</code>	<i>"No worries — I'll send you the notes afterwards."</i>	2 (laatste)
<code>chunk_1.wav</code>	<i>"Hey, are you coming to the three o'clock meeting?"</i>	0 (eerste)
<code>chunk_2.wav</code>	<i>"I can't — I've got a dentist appointment then."</i>	1

De werkelijke volgorde is **chunk_1** → **chunk_2** → **chunk_0**, dus `P = [2, 0, 1]`, en `prefix.json` bevat `[1, 2]`.

⚠ **P moet een geldige permutatie zijn:** lengte n , 0-geïndexeerd, elke waarde exact één keer. Als een waarde twee keer voorkomt, er eentje ontbreekt, of er waarden buiten het bereik (bijv. 1-geïndexeerd) zijn, levert dat een score van 0 op. Een onjuist opgemaakt bestand of een bestand dat geen JSON is, wordt geweigerd.

Beoordeling

De beoordelingsmaatstaf voor deze taak is **paarsgewijze ordeningsnauwkeurigheid**. Voor elk paar fragmenten wordt gevraagd: *welk van de twee moet als eerste komen?* Een paar is correct als je voorspelling hetzelfde antwoord geeft als de ground truth. Voor een dialoog met n fragmenten zijn er

$$M = n(n - 1)/2$$

paren; laat I het aantal inversies zijn — paren die anders geordend zijn dan in de ground truth:

$$\text{score} = 1 - \frac{I}{M}$$

$$0 \leq \text{score} \leq 1$$

i De eindscore is het gemiddelde van de scores per dialoog over alle dialogen in de split.

Toegestane modellen

Je mag alleen de volgende vooraf getrainde modellen (het fine-tunen van Qwen mag wel) gebruiken om deze taak op te lossen, zowel tijdens training als tijdens evaluatie. Al deze modellen zijn al gedownload en beschikbaar in de omgeving. In de baseline-notebook `solution.ipynb` kun je voorbeelden van het gebruik ervan bekijken. Houd er rekening mee dat je geen enkel ander model mag gebruiken en dat je programma geen internettoegang heeft.

- **Spraakrepresentaties: wav2vec 2.0.** De **Whisper encoder** mag ook als feature extractor worden gebruikt. [wav2vec-modelkaart](#)
- **Automatische spraakherkenning (ASR): OpenAI Whisper** (elke grootte). [Whisper-modelkaart](#)
- **Taalmodel: Qwen2.5-0.5B**, dat zero-shot of na fine-tuning op de meegeleverde `train-split` mag worden gebruikt. [Qwen-modelkaart](#) Houd er rekening mee dat de limiet van 10 minuten zowel alle training of fine-tuning die je tijdens de beoordeling uitvoert als het evalueren op de evaluatieset moet omvatten.

Indienen

Of opslaan op een manier dat je het later hopelijk terug kan vinden.

- Open `solution.ipynb` en voer alle cellen uit. Controleer of hierdoor `answers.json` in de werkmap wordt geschreven met een permutatie voor elke dialoog in `dataset/test_public/` (100 dialogen). Tijdens de beoordeling wordt de notebook opnieuw uitgevoerd op de verborgen testset en wordt de `answers.json` die daar wordt geproduceerd, beoordeeld.
- Verbeter de oplossing als je dat wilt — of niet; alleen de baseline volstaat om de pipeline te valideren.
- Open het tabblad Git in de linkerzijbalk van JupyterLab.
- **Stage** `solution.ipynb` (het +-pictogram ernaast).
- Voer een commitbericht in (voor jezelf, niemand gaat het lezen) en klik op **Commit**.
- Klik op het wolkpictogram met de omhoogwijzende pijl om te pushen.
- Ga terug naar deze wedstrijdpagina en klik op **Submit**.

Dien exact één bestand in, met de naam `solution.ipynb`. :)